Study Habits Survey: My Exploration Notebook

Name: *(Nahayo Jean de Dieu)*
Date: *(03/11/2026)*

Registration Number: *(224004771)*

1. Setup

```
x = 50
```

```
print("Hello, Jupyter!")
2 + 3 * 4
```

```
Hello, Jupyter!
14
```

```
x * 2
```

```
100
```

Why did `x * 2` give a different answer the second time?

At first `x` was 10, so `x * 2` gave 20. When I changed the cell to `x = 50` but did not run it, the kernel still remembered `x = 10`, so `x * 2` still gave 20. After I ran `x = 50`, the kernel stored the new value, so `x * 2` gave 100.

This shows that the kernel remembers the last value that was **run**, not what is written in the cell, so the order in which cells are run matters.

```
import pandas as pd
print('Pandas version:', pd.__version__)
```

```
Pandas version: 2.2.3
```

- `import pandas as pd` loads the pandas library, which gives Python tools for working with tables of data. I give it the short name `pd` so I can type `pd` instead of the full word `pandas` every time.
- `print('Pandas version:', pd.__version__)` displays the text "Pandas version:" followed by the version number of pandas installed here (2.2.3). It shows that the library loaded correctly.

2. About the dataset

Task 2: describe the dataset in your own words using Markdown: a heading, a short paragraph, a bullet list of columns and a table of at least 3 columns with their meaning.

Study Habits Survey Dataset

This dataset contains survey answers from **80 students** about their study habits. It is simulated data made for teaching, so it does not describe real people.

Columns in the dataset:

- `student_id`: unique ID of each student
- `faculty`: the student's faculty
- `year`: year of study (1 to 4)

- `age`: age in years
- `study_hours_per_week`: hours of study per week
- `sleep_hours`: average hours of sleep per night
- `commute_minutes`: minutes travelling to campus each day
- `phone_hours_per_day`: hours on the phone each day
- `gpa`: grade point average (0 to 4)

Column	Meaning	Type
faculty	Science, Business, Education, Engineering or Health	text
study_hours_per_week	Hours of study per week	number
gpa	Grade point average (0 to 4)	number

Tool used: Python with pandas.

3. Loading and first look

Task 3: load the file, then inspect it. After each output, write what you observe.

```
df = pd.read_csv('study_habits_survey.csv')
df.head()
```

	student_id	faculty	year	age	study_hours_per_week	sleep_hours	commute_minutes	phone_hours_per
0	S001	Science	4	23	17.1	6.3	14	
1	S002	Engineering	2	21	12.5	5.6	24	
2	S003	Engineering	1	20	16.3	5.8	17	
3	S004	Education	4	22	10.7	6.1	33	
4	S005	Education	3	23	12.2	8.9	5	

Next steps: Generate code with `df` New interactive sheet

What I observe: Each row is one student, and the columns show their faculty, year, age, study hours, sleep, commute, phone use and GPA. The first five rows look normal and nothing seems strange.

```
df.shape
```

```
(80, 9)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 80 entries, 0 to 79
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   student_id            80 non-null     object 
 1   faculty               80 non-null     object 
 2   year                  80 non-null     int64  
 3   age                   80 non-null     int64  
 4   study_hours_per_week  80 non-null     float64
 5   sleep_hours           80 non-null     float64
 6   commute_minutes       80 non-null     int64  
 7   phone_hours_per_day   80 non-null     float64
 8   gpa                   80 non-null     float64
dtypes: float64(4), int64(3), object(2)
memory usage: 5.8+ KB
```

```
df.describe()
```

	year	age	study_hours_per_week	sleep_hours	commute_minutes	phone_hours_per_day	
count	80.000000	80.000000	80.000000	80.000000	80.000000	80.000000	80.0
mean	2.612500	21.187500	13.387500	6.770000	36.512500	4.073750	2.5
std	1.130629	1.543412	4.693016	0.965008	19.836033	1.542294	0.3
min	1.000000	18.000000	3.300000	4.700000	5.000000	0.500000	1.8
25%	2.000000	20.000000	9.800000	6.100000	24.000000	3.000000	2.3
50%	3.000000	21.000000	13.100000	6.650000	35.500000	4.300000	2.5
75%	4.000000	22.000000	16.200000	7.225000	44.250000	5.125000	2.7
max	4.000000	24.000000	28.600000	9.000000	93.000000	7.500000	3.3

What I observe: Students study about 13 hours per week on average (the middle value is about 13.1), and the average GPA is about 2.58. The lowest GPA is 1.86 and the highest is 3.39.

4. Asking questions

Task 4: follow the pattern Question, Code, Output, Interpretation.

Q1. What is the average GPA of all students?

```
df['gpa'].mean()
```

```
np.float64(2.581)
```

Interpretation: *(one or two sentences)*

Q2. Which faculty studies the most hours per week on average?

```
df.groupby('faculty')['study_hours_per_week'].mean()
```

faculty	study_hours_per_week
Business	13.509091
Education	11.965000
Engineering	14.933333
Health	14.276923
Science	12.260000

dtype: float64

Interpretation:

Q3. How many students use their phone for more than 5 hours a day?

```
df[df['phone_hours_per_day'] > 5].shape[0]
```

```
24
```

Interpretation:

Q4. Are study hours and GPA related?

```
df[['study_hours_per_week', 'gpa']].corr()
```

	study_hours_per_week	gpa
study_hours_per_week	1.000000	0.571965
gpa	0.571965	1.000000

Interpretation:

Q5. My own question: *(write it here)*

```
df[['phone_hours_per_day', 'gpa']].corr()
```

	phone_hours_per_day	gpa
phone_hours_per_day	1.00000	-0.26063
gpa	-0.26063	1.00000

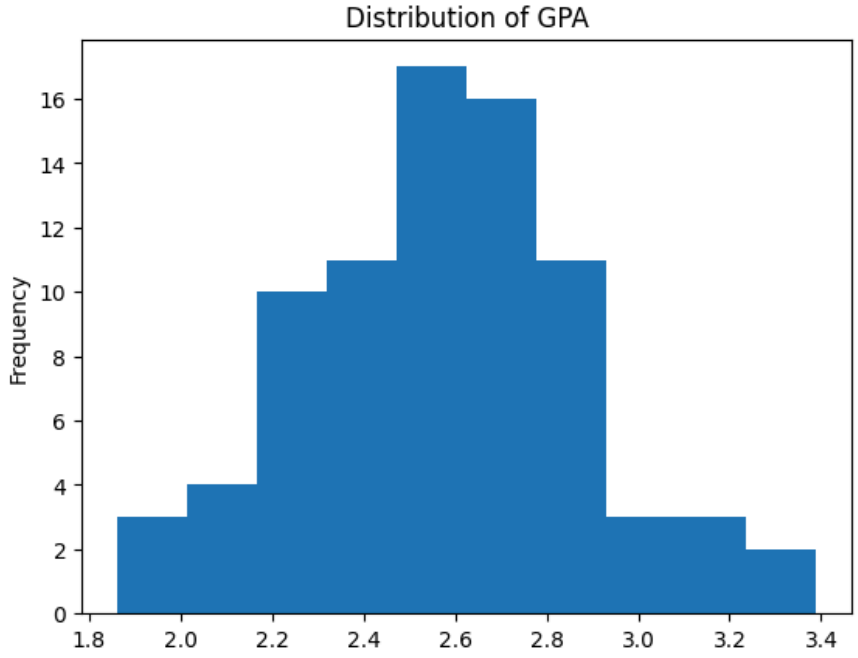
Interpretation: The correlation between phone hours per day and GPA is about -0.26, a weak negative relationship. Students who use their phone more tend to have slightly lower GPAs, but the link is weak and does not prove that phone use causes lower grades.

5. Charts

Task 5: make two simple charts and add a one-sentence caption under each.

```
df['gpa'].plot.hist(bins=10, title='Distribution of GPA')
```

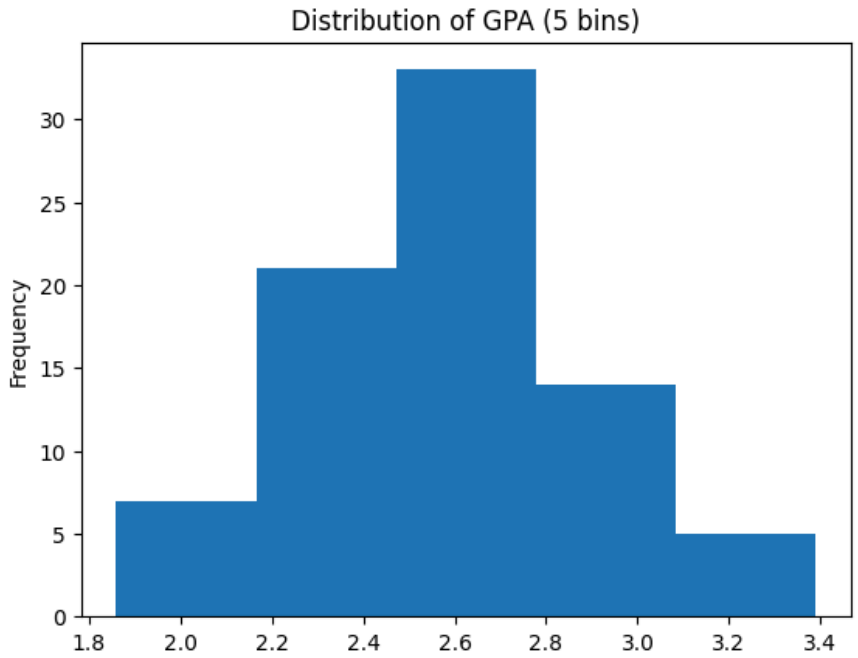
```
<Axes: title={'center': 'Distribution of GPA'}, ylabel='Frequency'>
```



Caption: Most students have a GPA between about 2.4 and 2.8, and only a few students have a GPA above 3.0.

```
df['gpa'].plot.hist(bins=5, title='Distribution of GPA (5 bins)')
```

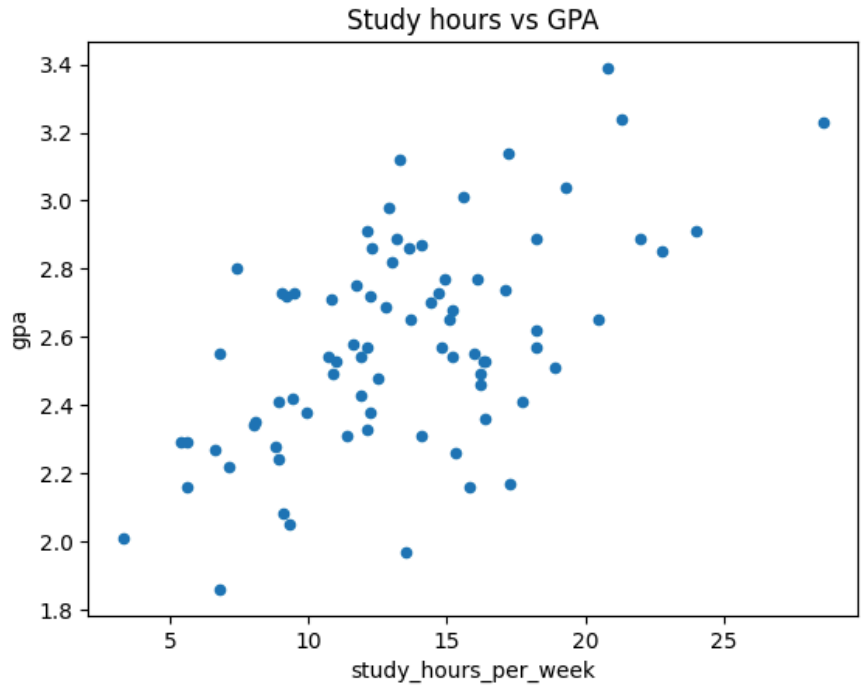
```
<Axes: title={'center': 'Distribution of GPA (5 bins)'}, ylabel='Frequency'>
```



With 5 bins the chart looks simpler and smoother, but it hides some detail about how GPAs are spread. With 10 bins I can see the shape more clearly.

```
df.plot.scatter(x='study_hours_per_week', y='gpa', title='Study hours vs GPA')
```

```
<Axes: title={'center': 'Study hours vs GPA'}, xlabel='study_hours_per_week', ylabel='gpa'>
```



Caption: The dots drift upward from left to right, so students who study more hours per week tend to have higher GPAs.

6. Make it reproducible

Task 6: use Kernel > Restart & Run All. Does everything still work from top to bottom? Note any fixes you made.

Result: I used Restart and run all, and the whole notebook ran from top to bottom without errors, so it is reproducible.

7. Conclusions

Task 7: write 3 to 5 bullet points that summarise what you learned about the students' study habits, and one limitation of this dataset.

-

Reflection: *What is the main advantage of a notebook over a plain Python script?*

7. Conclusions

- Students who spend more hours studying each week generally get higher GPAs. The correlation between study hours and GPA is about 0.57, which is a moderate positive link.
- Engineering students study the longest, about 14.9 hours a week, while Education students study the shortest, about 12.0 hours.
- Phone use is high in this group: 24 of the 80 students (30%) spend more than 5 hours a day on their phones.
- Students with more phone time tend to have slightly lower GPAs, but the link is weak (correlation about -0.26).

Limitation: The data is simulated and covers only 80 students, so the findings may not apply to real students. Also, a correlation shows that two things move together, but it does not prove that one causes the other.

Reflection: The main advantage of a notebook over a plain Python script is that the code, its results, the charts and my explanations are all in one document. I can also run the code in small steps and see each result immediately, which makes it easier to test ideas and explain my work.